

CAP 6. CAMADA DE REDE

AULA 9: ALGORITMOS DE ROTEAMENTO: LINK STATE E DISTANCE VECTOR

INE5422 REDES DE COMPUTADORES II

PROF. ROBERTO WILLRICH (INE/UFSC)

ROBERTO.WILLRICH@UFSC.BR

[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Algoritmos de roteamento

Algoritmos “link state”. Com Informação Global:

- Todos os roteadores têm informações completas da topologia e dos custos dos enlaces

Algoritmos “distance vector”. Com Informação Descentralizada:

- Roteadores só conhecem informações sobre seus vizinhos e os enlaces para eles
- Processo de computação iterativo
 - troca de informações com os vizinhos

Algoritmo de roteamento Link-State

Algoritmo de Dijkstra

- Topologia de rede e custo dos enlaces são conhecidos por todos os nós
- Implementado via “link state broadcast”
- Todos os nós (roteadores) têm a mesma informação
- Cada nó computa caminhos de menor custo deste nó para todos os outros nós
- Fornece uma tabela de roteamento para aquele nó
- Convergência: após k iterações conhece o caminho de menor custo para k destinos (nós no grafo)

Algoritmo de roteamento Link-State

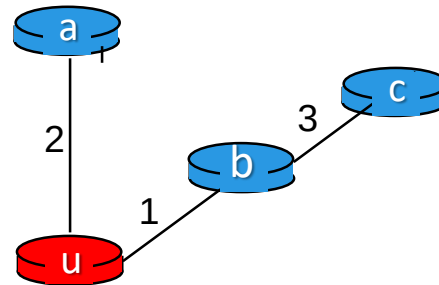
Notação no algoritmo de Dijkstra

- $C(i,j)$: custo do enlace do nó i ao nó j .
 - Custo é infinito se não houver ligação conhecida entre i e j
- $D(v)$: valor atual do custo do caminho da fonte ao destino v
- $P(v)$: nó predecessor ao longo do caminho da fonte ao nó v , isto é, antes do v
- N' : conjunto de nós cujo caminho de menor custo é definitivamente conhecido

Algoritmo de Dijkstra

1 Inicialização (rodando em um nó u):

- 2 $N' = \{u\}$
- 3 para todos os nós v
- 4 se v é adjacente a u
- 5 então $D(v) = c(u,v)$
- 6 senão $D(v) = \infty$



```
2.  $N' = \{u\}$ 
3-6:  $D(a) = 2; D(b) = 1; D(c) = \infty;$ 
9:  $w = b$ 
10:  $N' = \{u, b\}$ 
11-12:  $D(c) = \min(\infty, 1+3) = 4$ 
9:  $w = a$ 
10:  $N' = \{u, b, a\}$ 
11: a não tem adjacente
9:  $w = c$ 
10:  $N' = \{u, b, a, c\}$ 
```

No roteador u:

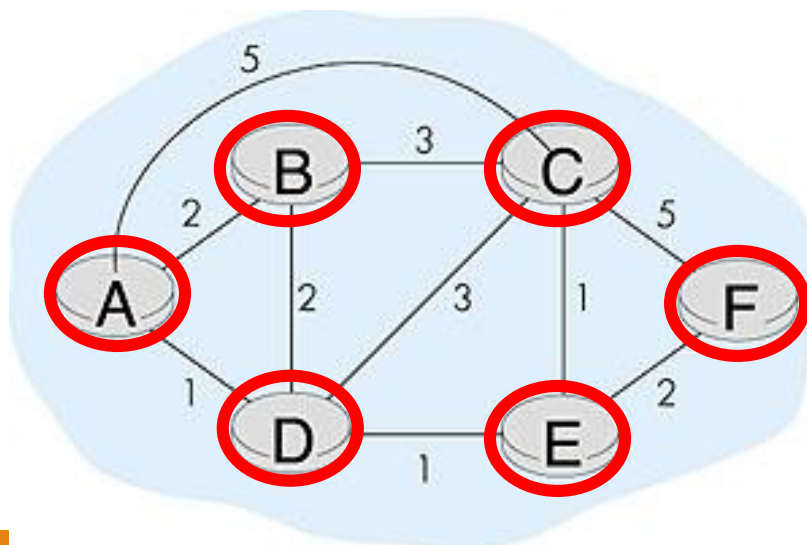
- $D(u) = 0; D(a) = 2; D(b) = 1;$
 $D(c) = 4$

8 Loop

- 9 ache w não em N' tal que $D(w)$ é um mínimo
- 10 acrescente w a N'
- 11 atualize $D(v)$ para todo v adjacente a w e não em N' :
- 12 $D(v) = \min(D(v), D(w) + c(w,v))$
- 13 /* novo custo para v é ou o custo anterior para v ou o menor
- 14 custo do caminho conhecido para w mais o custo de w a v */
- 15 até que todos os nós estejam em N'

Exemplo: Algoritmo de Dijkstra

Passo	N'	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
→ 0	A	2,A	5,A	1,A	∞	∞
→ 1	AD	2,A	4,D		2,D	∞
→ 2	ADE	2,A	3,E			4,E
→ 3	ADEB		3,E			4,E
→ 4	ADEBC					4,E
→ 5	ADEBCF					



Algoritmo de Dijkstra: exemplo

Passo	N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	∞	∞
1	AD	2,A	4,D		2,D	∞
2	ADE	2,A	3,E			4,E
3	ADEB		3,E			4,E
4	ADEBC					4,E
5	ADEBCF					

Árvore de caminhos mínimos
resultante originada em A:

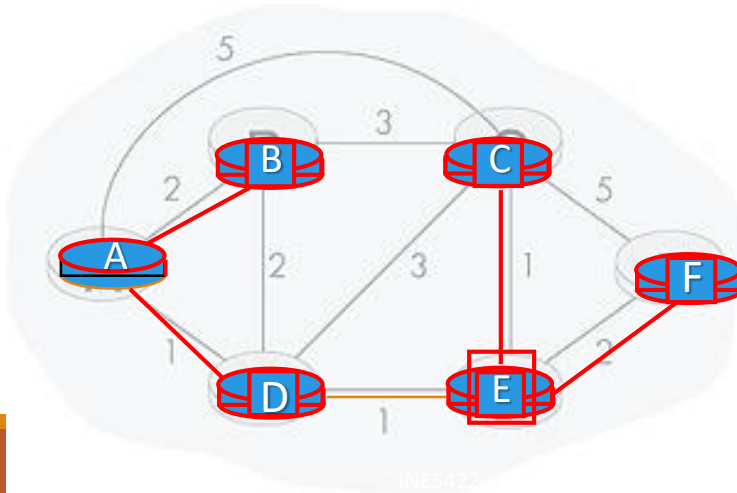


Tabela de encaminhamento resultante em A:

destino	enlace
B	(A,B)
C	(A,D)
D	(A,D)
E	(A,D)
F	(A,D)

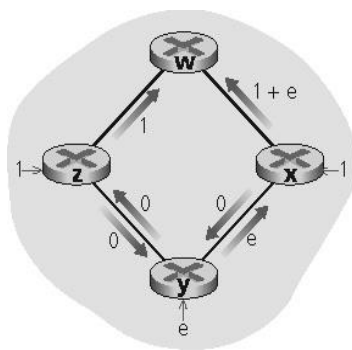
Discussão do algoritmo de Dijkstra

Complexidade do algoritmo: n nós

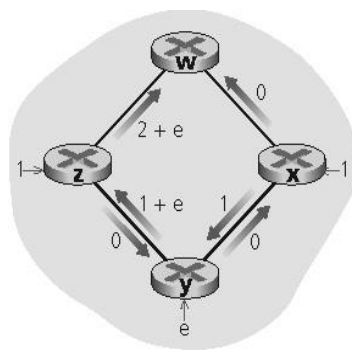
- Cada iteração: precisa verificar todos os nós w , que não estão em N'
- 1a interação n nós, 2a interação $n-1$ nós, 3a interação $n-2$ nós,...
- Número de nós buscados em todas as interações: $n(n+1)/2$
- Complexidade: $O(n^2)$

Oscilações possíveis:

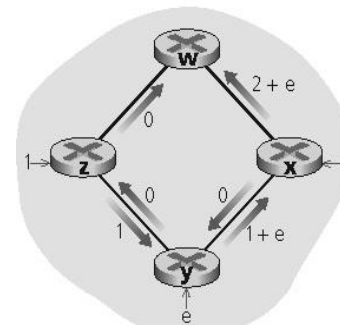
- Ex.: custo do enlace = quantidade de tráfego transportado



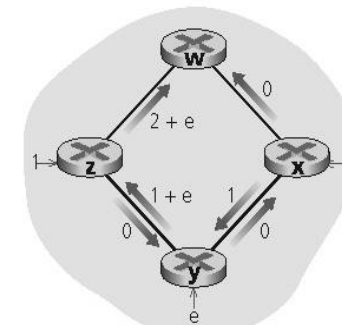
a. Roteamento inicial



b. x, y detectam melhor caminho até w em sentido horário



c. x, y, z detectam melhor caminho até w em sentido anti-horário

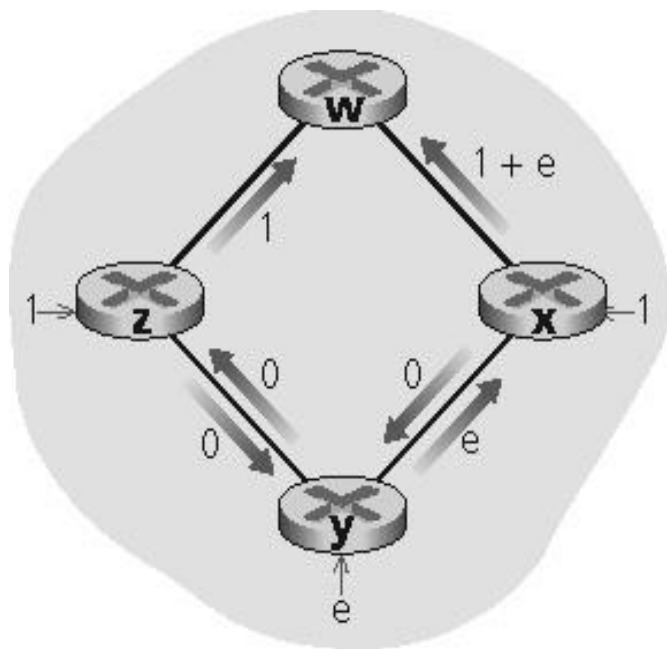


d. x, y, z detectam melhor caminho até w em sentido horário

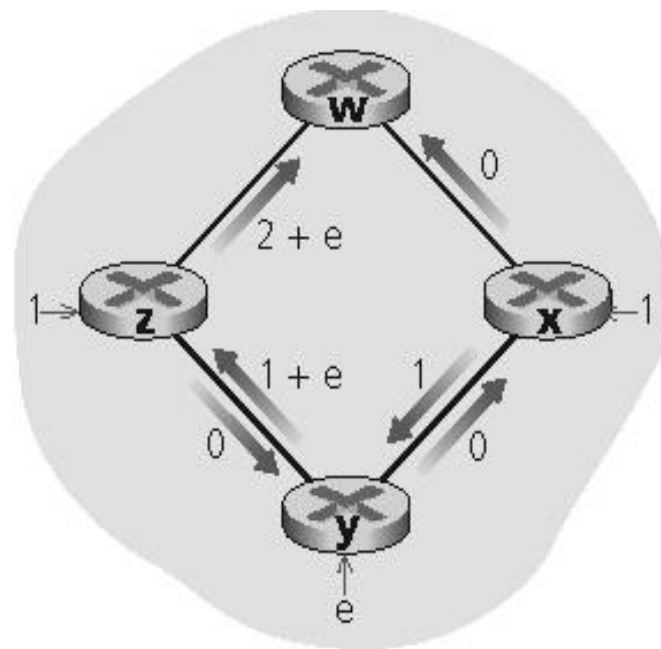
Discussão do algoritmo de Dijkstra

Oscilações possíveis:

- Ex.: custo do enlace = quantidade de tráfego transportado



a. Roteamento inicial

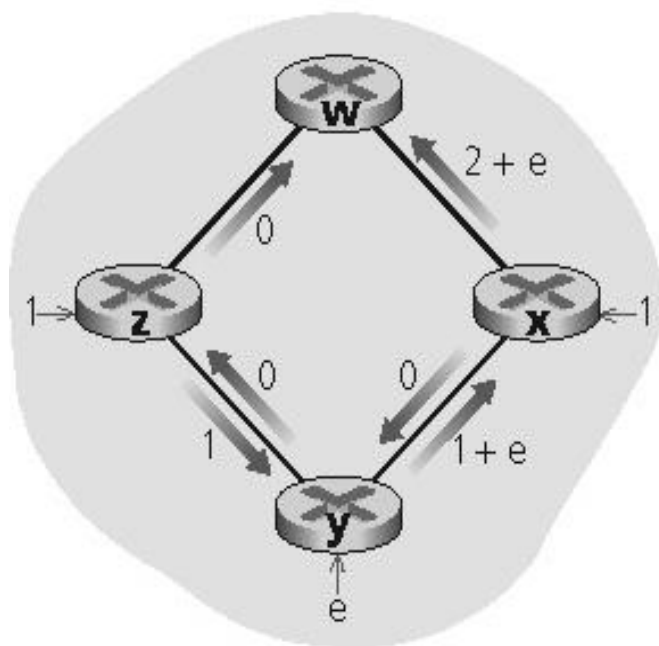


b. x, y detectam melhor caminho até w em sentido horário

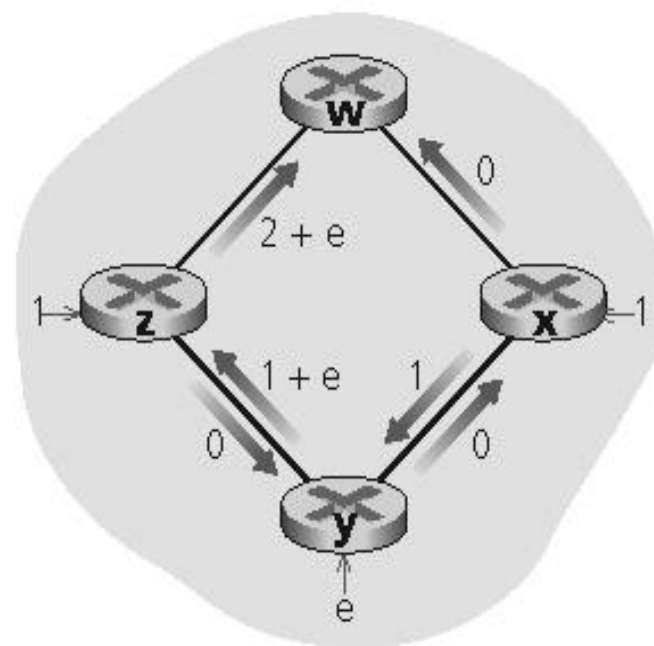
Discussão do algoritmo de Dijkstra

Oscilações possíveis:

- Ex.: custo do enlace = quantidade de tráfego transportado



c. x, y, z detectam melhor caminho até w em sentido anti-horário

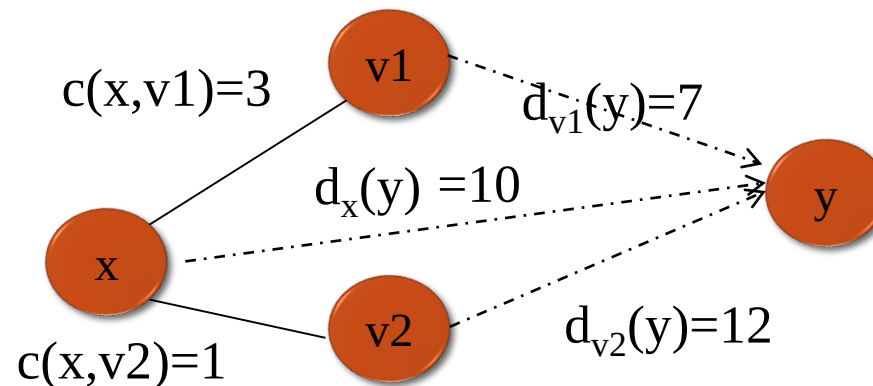


d. x, y, z detectam melhor caminho até w em sentido horário

Algoritmo vetor de distância (1)

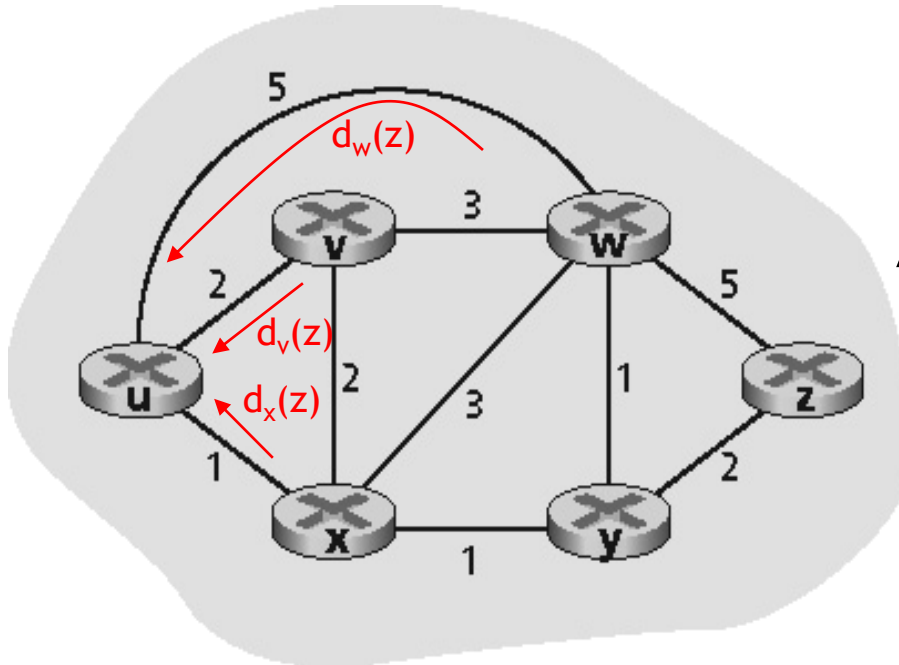
Equação de Bellman-Ford (programação dinâmica)

- Define
 - $d_x(y)$ = custo do caminho de menor custo de x para y
- Então
 - $d_x(y) = \min_v \{c(x,v) + d_v(y)\}$
 - Em que $\min_v$ é calculado para todos os vizinhos de x
 - No exemplo: $d_x(y) = \min_v \{c(x,v1) + d_{v1}(y), c(x,v2) + d_{v2}(y)\}$



Exemplo: Bellman-Ford (2)

Exemplo: custo entre o nó u e o nó z



U recebe a informação dos vizinhos:

$$d_v(z) = 5, d_x(z) = 3, d_w(z) = 3$$

A equação B-F diz que:

$$\begin{aligned} d_u(z) &= \min \{ c(u,v) + d_v(z), \\ &\quad c(u,x) + d_x(z), \\ &\quad c(u,w) + d_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

Caminho mínimo $d_u(z) = 4$
passando por x

O nó que atinge o mínimo é o próximo salto no caminho mais curto
→ tabela de roteamento

Algoritmo vetor de distância (3)

Define a forma de comunicação de vizinho para vizinho

- Atualizando a tabela de roteamento para que o pacote seja enviado para o vizinho no caminho de menor custo

Cada nó x mantém os seguintes dados de roteamento

- Para cada vizinho v , ele mantém o custo $c(x,v)$
- O vetor de distâncias (DV) do nó x
 - $D_x = [D_x(y) : y \in N]$
- Os vetores de distância de seus vizinhos
 - Para cada vizinho v , x mantém $D_v = [D_v(y) : y \in N]$

Algoritmo vetor de distância (4)

Idéia básica:

- Cada nó envia periodicamente sua própria estimativa de vetor de distâncias aos vizinhos
- Quando o nó x recebe nova estimativa de DV do vizinho, ele atualiza seu próprio DV usando a equação B-F:
 - $D_x(y) = \min_v \{c(x,v) + D_v(y)\}$ para cada nó $y \in N$
- Ao menos em condições naturais, a estimativa $D_x(y)$ converge para o menor custo atual $dx(y)$

Algoritmo vetor de distância

Tabela do nó x

De	Custo até		
	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

DV do Nó

DVs dos vizinhos

Tabela do nó y

De	Custo até		
	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

DV do Nó

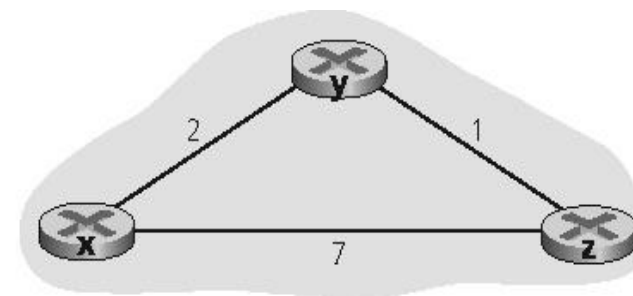
DVs dos vizinhos

Tabela do nó z

De	Custo até		
	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

DVs dos vizinhos

DV do nó



Algoritmo vetor de distância

Tabela do nó x

		Custo até		
De		x	y	z
	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

Tabela do nó y

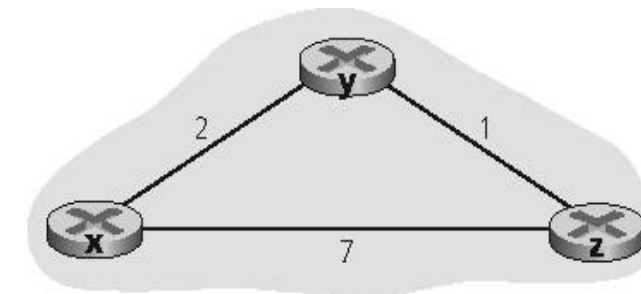
		Custo até		
De		x	y	z
	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

Tabela do nó z

		Custo até		
De		x	y	z
	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} \\ = \min\{2+1, 7+0\} = 3$$



Tempo

Algoritmo vetor de distância

Tabela do nó x

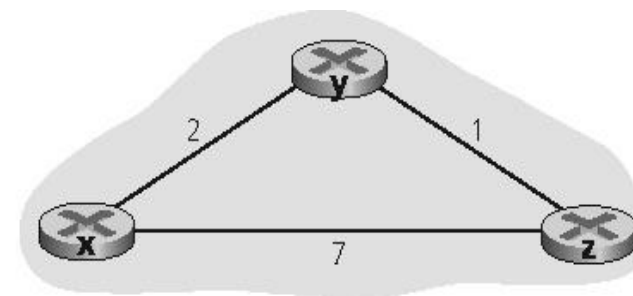
		Custo até		
		x	y	z
De	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

Tabela do nó y

		Custo até		
		x	y	z
De	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

Tabela do nó z

		Custo até		
		x	y	z
De	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0



.....→ Tempo

Algoritmo vetor de distância (5)

Iterativo, assíncrono: cada iteração local é causada por:

- Mudança no custo do enlace local
- Mensagem de atualização DV do vizinho

Distribuído:

- Cada nó notifica os vizinhos apenas quando seu DV mudar
- Os vizinhos então notificam seus vizinhos, se necessário

Cada nó:

espera por (mudança no custo do enlace local na mensagem do vizinho)

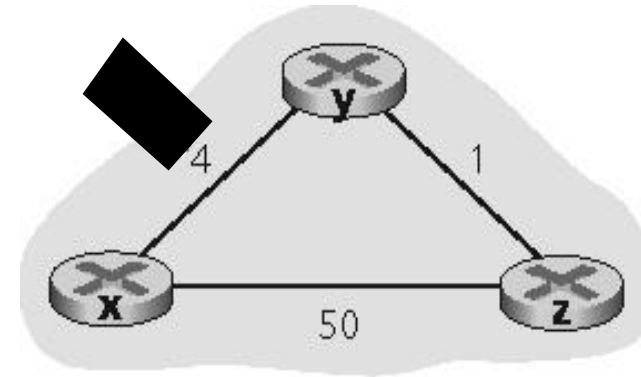
recalcula estimativas

se o DV para qualquer destino mudou, **notifica** os vizinhos

Vetor de distância: mudanças no custo do enlace

Mudanças no custo do enlace:

- Nó detecta mudança no custo do enlace local
- Atualiza informações de roteamento, recalcula o vetor de distância
- Se o DV muda, notifica vizinhos



a.

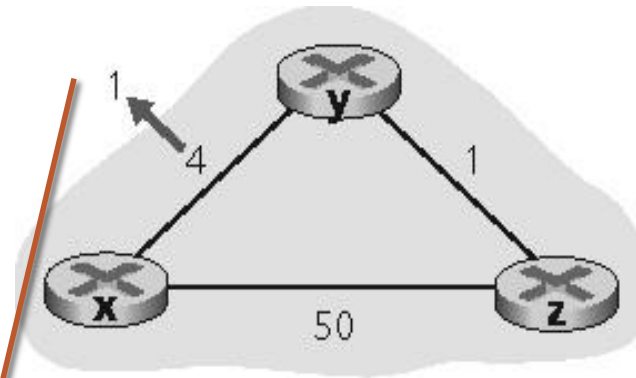
Até				
	x	y	z	
De x	0	4	5	
y	4	0	1	
z	5	1	0	
Nó X (inicial)				

Até				
	x	y	z	
x	0	4	5	
y	4	0	1	
z	5	1	0	
Nó y (inicial)				

Vetor de distância: mudanças no custo do enlace

Mudanças no custo do enlace:

- Nó detecta mudança no custo do enlace local
- Atualiza informações de roteamento, recalcula o vetor de distância
- Se o DV muda, notifica vizinhos



a.

Até				
	x	y	z	
x	0	4	5	
De y	4	0	1	
z	5	1	0	

Nó X (inicial)

Até				
	x	y	z	
x	0	4	5	
De y	1	0	1	
z	5	1	0	

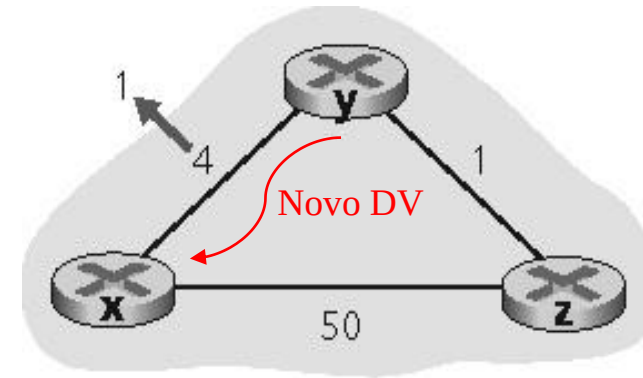
Nó y (custo reduzido)

$$D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} \\ = \min\{1+0, 1+50\} = 1$$

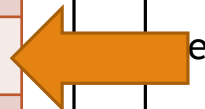
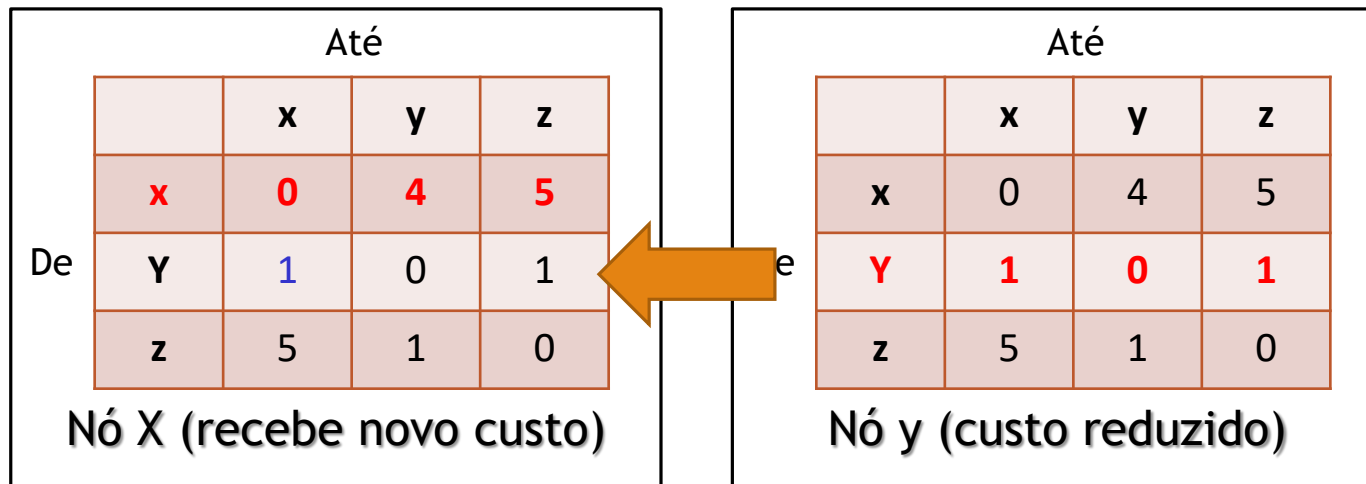
Vetor de distância: mudanças no custo do enlace

Mudanças no custo do enlace:

- Nó detecta mudança no custo do enlace local
- Atualiza informações de roteamento, recalcula o vetor de distância
- Se o DV muda, notifica vizinhos



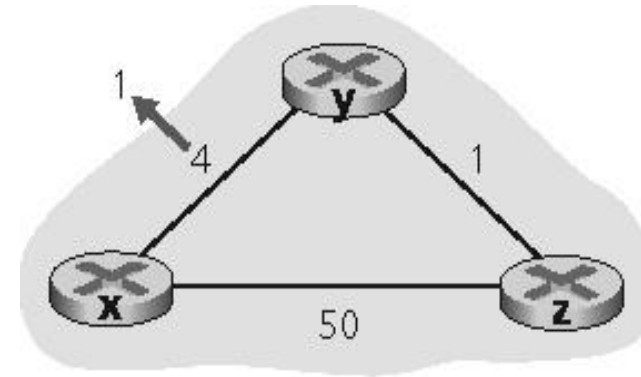
a.



Vetor de distância: mudanças no custo do enlace

Mudanças no custo do enlace:

- Nó detecta mudança no custo do enlace local
- Atualiza informações de roteamento, recalcula o vetor de distância
- Se o DV muda, notifica vizinhos



a.

“boas notícias viajam depressa”

Até				
	x	y	z	
De	x	0	1	2
	y	1	0	1
	z	5	1	0
Nó X (recalcula DV)				

Até				
	x	y	z	
De	x	0	4	5
	y	1	0	1
	z	5	1	0
Nó y (custo reduzido)				

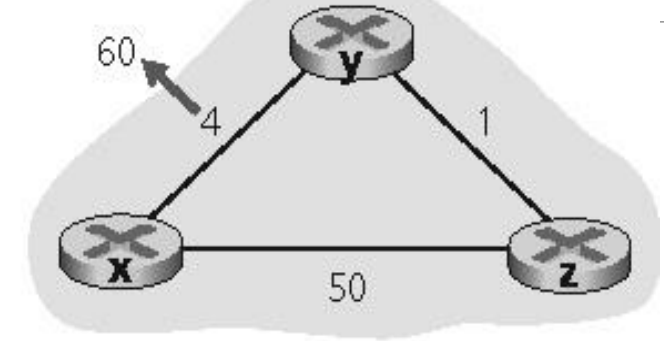
$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min\{1 + 0, 50 + 1\} = 1$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} \\ = \min\{1 + 1, 50 + 0\} = 2$$

Vetor de distância: mudanças no custo do enlace

Mudanças no custo do enlace:

- Boas notícias viajam rápido
- Más notícias viajam devagar — problema da “contagem ao infinito”!



b.

$$\begin{aligned} D_y(x) &= \min\{c(y,x) + D_x(x), \\ &\quad c(y,z) + D_z(x)\} \\ &= \min\{60+0, 1+5\} \\ &= 6 \text{ passando por } z \end{aligned}$$

No início (antes da mudança do custo)

- $D_y(x)=4, D_y(z)=1, D_z(y)=1, D_z(x)=5$

Até				
De		x	y	z
	x	0	4	5
	y	4	0	1
	z	5	1	0
Nó x (inicial)				

Até				
De		x	y	z
	x	0	4	5
	y	4	0	1
	z	5	1	0
Nó y (inicial)				

Até				
De		x	y	z
	x	0	4	5
	y	4	0	1
	z	5	1	0
Nó z (inicial)				

Vetor de distância: mudanças no custo do enlace

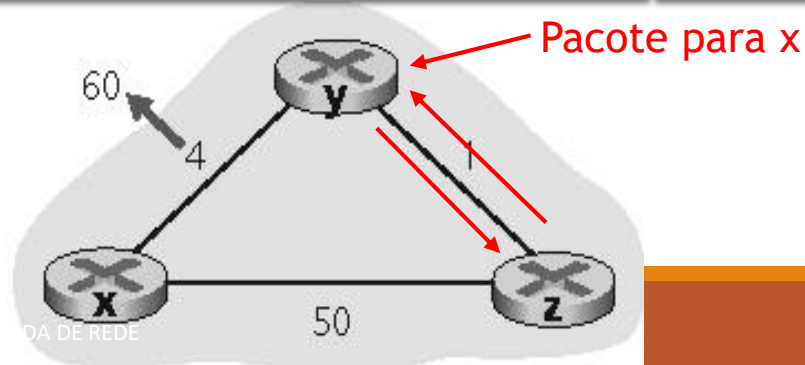
Y detecta a mudança de custo

- $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60, 6\} = 6$
- Um equivoco! Gera um loop de roteamento

		Até			
		x	y	z	
De	x	0	4	5	
	y	4	0	1	
	z	5	1	0	
Nó x (inicial)					

		Até			
		x	y	z	
De	x	0	4	5	
	y	6 (z)	0	1	
	z	5 (y)	1	0	
Nó y (detecta mudança)					

		Até			
		x	y	z	
De	x	0	4	5	
	y	4	0	1	
	z	5 (y)	1	0	
Nó z (inicial)					



Vetor de distância: mudanças no custo do enlace

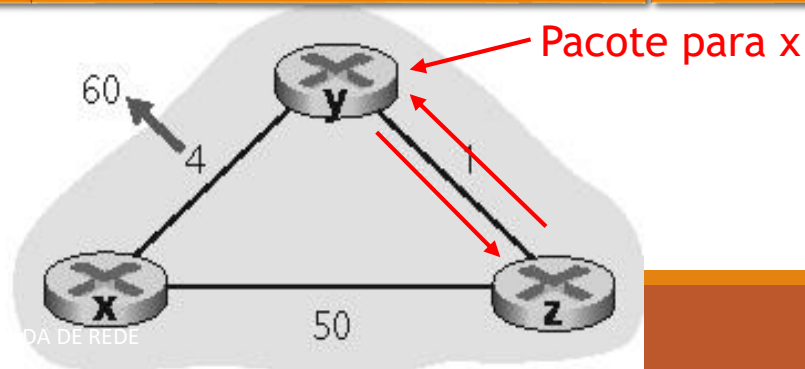
Atualizando DV, y informa a z

- Y informa que $D_y(x)=6$ e como z sabe que $c(z,y)=1$, então $D_z(x)=7$
- Z informa novo DV a y

		Até			
		x	y	z	
De	x	0	4	5	
	y	4	0	1	
	z	5	1	0	
Nó x (inicial)					

		Até			
		x	y	z	
De	x	0	4	5	
	y	6 (z)	0	1	
	z	5	1	0	
Nó y (detecta mudança)					

		Até			
		x	y	z	
De	x	0	4	5	
	y	6 (z)	0	1	
	z	7 (y)	1	0	
Nó z (atualizando)					



Vetor de distância: mudanças no custo do enlace

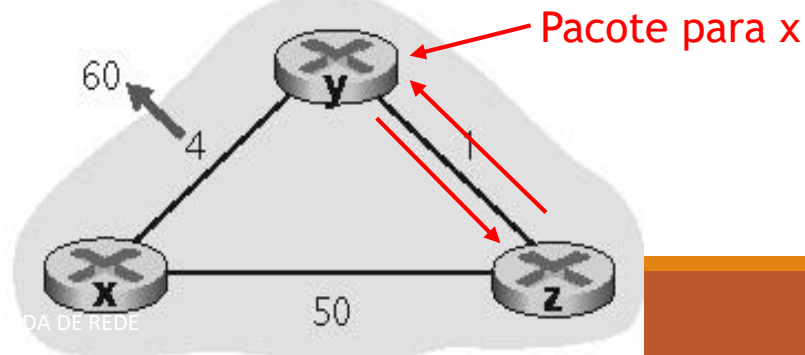
Atualizando DV, z informa a y

- z informa que $D_z(x)=7$ e como y sabe que $c(y,z)=1$, então $D_y(x)=8$
- y informa novo DV a z

Nó x (inicial)				
Até				
	x	y	z	
De x	0	4	5	
De y	4	0	1	
De z	5	1	0	

Nó y (atualizando)				
Até				
	x	y	z	
De x	0	4	5	
De y	8 (z)	0	1	
De z	7 (y)	1	0	

Nó z (atualizado)				
Até				
	x	y	z	
De x	0	4	5	
De y	6 (z)	0	1	
De z	7 (y)	1	0	



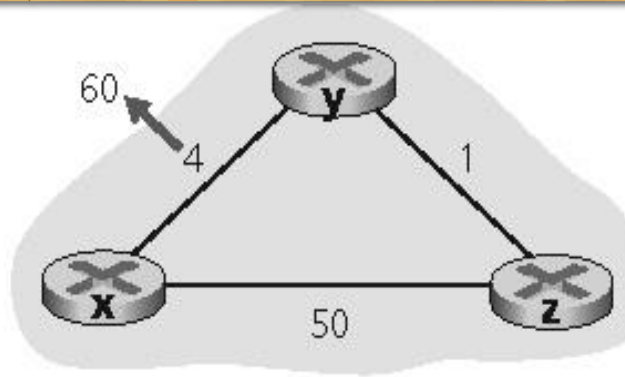
Vetor de distância: mudanças no custo do enlace

44 iterações antes de o algoritmo estabilizar!!!

Até				
De		x	y	z
	x	0	4	5
	y	4	0	1
	z	5	1	0
Nó x (inicial)				

Até				
De		x	y	z
	x	0	4	5
	y	8 (z)	0	1
	z	7 (y)	1	0
Nó y (atualizado)				

Até				
De		x	y	z
	x	0	4	5
	y	8 (z)	0	1
	z	7 (y)	1	0
Nó z (atualizando)				



b.

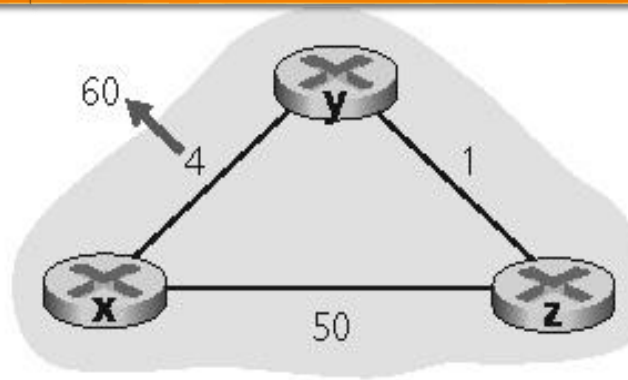
Vetor de distância: mudanças no custo do enlace

44 iterações antes de o algoritmo estabilizar!!!

Até				
De		x	y	z
	x	0	51	50
	y	51	0	1
	z	50	1	0
Nó x (inicial)				

Até				
De		x	y	z
	x	0	51	50
	y	51	0	1
	z	50	1	0
Nó y (atualizado)				

Até				
De		x	y	z
	x	0	51	50
	y	51	0	1
	z	50	1	0
Nó z (atualizando)				



b.

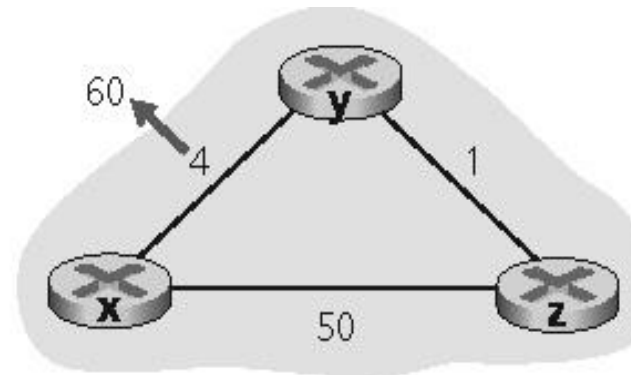
Vetor de distância: mudanças no custo do enlace

Solução: Reversão envenenada:

- Se Z roteia por Y para alcançar X :
 - Z diz a Y que sua distância (de Z) para X é infinita (então Y não roteará até X via Z)

Isso resolverá completamente o problema da contagem ao infinito?

- Não é solução geral para loops envolvendo três ou mais nós



b.

Comparação dos algoritmos LS e VD

Complexidade

- LS: com n nós, E links, $O(NE)$ mensagens enviadas
- DV: trocas somente entre vizinhos - Tempo de convergência varia

Tempo de convergência

- LS: algoritmo $O(N^2)$ exige mensagens $O(NE)$
 - Pode ter oscilações
- DV: tempo de convergência varia
 - Pode haver loops de roteamento durante a convergência
 - Problema da contagem ao infinito

Comparação dos algoritmos LS e VD

Robustez: o que acontece se um roteador funciona mal?

- LS:
 - Nós podem informar custos de link incorretos
 - Cada nó calcula sua própria tabela de roteamento: aumenta a robustez
- DV:
 - Nó DV pode informar custo de caminho incorreto
 - Tabela de cada nó é usada por outros
 - Propagação de erros pela rede

Pontos Importantes

Algoritmo Link State e Distance Vector

- Entender princípios gerais
- Saber comparar algoritmos: vantagens e deficiências